**LAPORAN PRAKTIKUM**

**STRUKTUR DATA**

**STACK**


**Disusun Oleh:**

Gina Ramadhani

Nim: 2511533014

Dosen Pengampu: DR. Wahyudi, S.T, M.T

Asisten Praktikum: Sherly Sukmadira

**DAPERTEMEN INFORMATIKA**

**FAKULTAS TEKNOLOGI INFORMASI**

**UNIVERSITAS ANDALAS**


**TAHUN 2026**

# KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum dengan judul "*Stack*" ini dengan baik. Laporan ini disusun sebagai salah satu bentuk tugas praktikum yang bertujuan untuk memahami konsep struktur data dalam pemrograman, khususnya mengenai implementasi Stack dalam bahasa Java.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan baik dari segi penulisan maupun isi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang. Penulis juga mengucapkan terima kasih kepada dosen pengampu dan semua pihak yang telah membantu dalam pelaksanaan praktikum serta penyusunan laporan ini.

Padang, 2026

Penulis

# DAFTAR ISI

**KATA PENGANTAR** ..... **I**
**DAFTAR ISI** ..... **II**
**BAB I PENDAHULUAN** ..... **1**
1.1 Latar Belakang ..... 1
1.2 Tujuan Praktikum ..... 1
1.3 Manfaat Praktikum ..... 1
**BAB II PEMBAHASAN** ..... **2**
2.1 Dasar Teori ..... 2
2.2 Praktikum “NilaiMaksimum_2511533014” ..... 3
2.3 Praktikum “latihanStack_2511533014” ..... 6
2.4 Praktikum “StackArray_2511533014” ..... 8
2.5 Praktikum “stackArrayDriver” ..... 11
2.6 Praktikum “StackPostfix_2511533014” ..... 14
2.7 Praktikum “SiswaStack_2511533014” ..... 16
2.8 Latihan Praktikum ..... 20
**BAB III PENUTUP** ..... **26**
3.1 Kesimpulan ..... 26
**DAFTAR PUSTAKA** ..... **27**

# BAB I
# PENDAHULUAN

## 1.1 Latar Belakang

Dalam pemrograman, struktur data memiliki peran penting dalam mengatur dan mengelola data secara efisien. Salah satu struktur data dasar yang sering digunakan adalah Stack, yaitu struktur data yang bekerja dengan prinsip *Last In First Out (LIFO)*, di mana data yang terakhir dimasukkan akan menjadi data pertama yang dikeluarkan. Stack banyak digunakan dalam berbagai kasus seperti manajemen memori, proses undo-redo, serta evaluasi ekspresi dalam pemrograman. Oleh karena itu, melalui praktikum ini mahasiswa mempelajari konsep Stack serta implementasinya menggunakan bahasa pemrograman Java agar dapat memahami cara kerja dan penerapannya dalam pengolahan data.

## 1.2 Tujuan Praktikum

1. Memahami konsep dasar struktur data **Stack**.
2. Mengetahui prinsip kerja Stack yaitu *Last In First Out (LIFO)*.
3. Mengimplementasikan operasi dasar Stack seperti *push*, *pop*, dan *peek*.

## 1.3 Manfaat Praktikum

1. Mahasiswa memahami cara kerja struktur data Stack.
2. Meningkatkan kemampuan dalam mengelola data menggunakan metode LIFO.
3. Melatih logika pemrograman dalam mengimplementasikan operasi Stack.

# BAB II
# PEMBAHASAN

## 2.1 Dasar Teori

Stack merupakan salah satu struktur data linear yang bekerja berdasarkan prinsip Last In First Out (LIFO), yaitu elemen yang terakhir dimasukkan ke dalam struktur akan menjadi elemen pertama yang dikeluarkan. Konsep ini dapat dianalogikan seperti tumpukan benda, misalnya tumpukan buku, di mana buku yang terakhir diletakkan di atas akan menjadi yang pertama diambil. Dalam pemrograman, Stack digunakan untuk mengatur data secara berurutan dengan akses terbatas hanya pada satu sisi, yaitu bagian atas (*top*).

Operasi dasar pada Stack meliputi push, pop, dan peek. Operasi *push* digunakan untuk menambahkan elemen ke dalam Stack, *pop* digunakan untuk menghapus elemen teratas dari Stack, sedangkan *peek* digunakan untuk melihat elemen teratas tanpa menghapusnya. Selain itu, terdapat juga operasi pendukung seperti *isEmpty* untuk mengecek apakah Stack kosong dan *isFull* (pada implementasi tertentu) untuk mengecek apakah Stack sudah penuh.

Dalam bahasa pemrograman Java, Stack dapat diimplementasikan menggunakan class bawaan Stack yang terdapat pada Java Collections Framework atau menggunakan struktur lain seperti Array maupun ArrayList. Penggunaan Stack sangat luas dalam dunia pemrograman, seperti dalam proses undo-redo, evaluasi ekspresi matematika (infix, postfix, prefix), pengecekan keseimbangan tanda kurung, serta pemanggilan fungsi secara rekursif.

## 2.2 Praktikum “NilaiMaksimum_2511533014”

```java
package pekan3_2511533014;
import java.util.Stack;

public class NilaiMkasimum_2511533014 {
    public static int max_2511533014 (Stack<Integer> s) {
        Stack<Integer>backup = new Stack<Integer>();
        int maxValue_2511533014 = s.pop();
        backup.push(maxValue_2511533014);
        while (!s.isEmpty()) {
            int next = s.pop();
            backup.push(next);
            maxValue_2511533014 = Math.max(maxValue_2511533014, next);
        }

        while (!backup.isEmpty()) {
            s.push(backup.pop());
        }

        return maxValue_2511533014;
    }

    public static void main(String[] args) {
        Stack<Integer> s = new Stack<Integer>();
        s.push(70);
        s.push(12);
        s.push(20);
        System.out.println("isi stack " + s);
        System.out.println("Stack Teratas " + s.peek());
        System.out.println("Nilai maksimum " + max_2511533014(s));
    }
```

Gambar 2.1 Kode Program Praktikum NilaiMaksimum

Kode program tersebut merupakan implementasi struktur data Stack di Java yang digunakan untuk mencari nilai maksimum dari elemen yang ada di dalam stack tanpa menghilangkan isi aslinya. Method max_2511533014 bekerja dengan memindahkan sementara isi stack ke stack cadangan untuk membandingkan setiap elemen dan menentukan nilai terbesar, kemudian mengembalikan isi stack ke kondisi semula. Pada method main, program mengisi stack dengan beberapa nilai, menampilkan isi stack, elemen teratas, serta nilai maksimum yang berhasil ditemukan.

Berikut Penjelasan langkah-langkah kerjanya:

1. package pekan3_2511533014;

   → Menentukan package tempat class berada.

2. import java.util.Stack;

   → Mengimpor class Stack dari Java Collections Framework.

3. Method Mencari Nilai Maksimum
    - public static int max_2511533014 (Stack<Integer> s) {
      → Method untuk mencari nilai maksimum dalam stack.
    - Stack<Integer> backup = new Stack<Integer>();
      → Membuat stack cadangan untuk menyimpan sementara data.
    - int maxValue_2511533014 = s.pop();
      → Mengambil elemen pertama dari stack sebagai nilai awal maksimum.
    - backup.push(maxValue_2511533014);
      → Menyimpan nilai tersebut ke stack cadangan.
    - while (!s.isEmpty()) {
      → Perulangan selama stack masih memiliki elemen.
    - int next = s.pop();
      → Mengambil elemen berikutnya dari stack.
    - backup.push(next);
      → Menyimpan elemen ke stack cadangan.
    - maxValue_2511533014 = Math.max(maxValue_2511533014, next);
      → Membandingkan nilai saat ini dengan nilai maksimum dan menyimpan yang terbesar.
4. Mengembalikan Isi Stack
    - while (!backup.isEmpty()) {
      → Perulangan untuk mengembalikan isi stack.
    - s.push(backup.pop());
      → Mengembalikan elemen dari stack cadangan ke stack asli.
    - return maxValue_2511533014;
      → Mengembalikan nilai maksimum.
5. Method Main
    - public static void main(String[] args) {
      → Method utama program.
    - Stack<Integer> s = new Stack<Integer>();
      → Membuat objek stack.
    - Baris 24-26

s.push(70);

s.push(12);

s.push(20);

→ Menambahkan elemen ke dalam stack.

- System.out.println("isi stack " + s);

  → Menampilkan seluruh isi stack.

- System.out.println("Stack Teratas " + s.peek());

  → Menampilkan elemen paling atas.

- System.out.println("Nilai maksimum " + max_2511533014(s));

  → Menampilkan nilai maksimum dalam stack.

Dari Langkah-langkah diatas kita akan mendapatkan output seperti gambar 2.2 dibawah ini.

```
<terminated> NilaiMkasimum_2511533014
isi stack [70, 12, 20]
Stack Teratas 20
Nilai maksimum 70
```

Gambar 2.2 Output dari kode program NilaiMaksimum

### 2.2.1 Analisis

Output tersebut menunjukkan bahwa program berhasil menjalankan operasi pada struktur data Stack dengan benar. Pada bagian *"isi stack [70, 12, 20]"*, terlihat bahwa elemen dimasukkan menggunakan metode *push* secara berurutan sehingga elemen terakhir (20) berada di posisi paling atas. Hal ini dibuktikan pada output *"Stack Teratas 20"* yang diambil menggunakan method *peek()*. Selanjutnya, program menghitung nilai maksimum dari seluruh elemen dalam stack melalui method max_2511533014, dan menghasilkan *"Nilai maksimum 70"*, yang merupakan nilai terbesar dari data yang ada. Proses pencarian maksimum juga tidak mengubah isi stack karena setelah operasi selesai, data tetap utuh seperti semula. Secara keseluruhan,

program berjalan dengan baik dan sesuai dengan konsep Stack yaitu Last In First Out (LIFO).

### 2.3 Praktikum "latihanStack_2511533014"

```
1  package pekan3_2511533014;
2  import java.util.Stack;
3  public class latihanStack_2511533014 {
4      public static void main (String[] args) {
5          Stack<Integer> s = new Stack<Integer>();
6          s.push(42);
7          s.push(-3);
8          s.push(17);
9          System.out.println ("nilai stack= "+s);
10         System.out.println ("nilai pop = "+s.pop());
11         System.out.println ("nilai stack stelah pop= "+s);
12         System.out.println ("nilai peek"+s.peek());
13         System.out.println ("nilai stack setelah peek= "+s);
14     }
15
16 }
```

Gambar 2.3 Kode Program praktikum latihanStack

Kode program tersebut merupakan implementasi sederhana struktur data **Stack** di Java yang digunakan untuk menunjukkan operasi dasar seperti *push*, *pop*, dan *peek*. Program menambahkan beberapa elemen ke dalam stack, kemudian menampilkan isi stack, menghapus elemen teratas menggunakan *pop*, serta melihat elemen teratas tanpa menghapusnya menggunakan *peek*. Melalui program ini, dapat dipahami bagaimana cara kerja Stack yang mengikuti prinsip *Last In First Out (LIFO)*.

Berikut penjelasan langkah-langkah kerjanya:

1. import java.util.Stack;

   → Mengimpor class Stack dari Java.

2. Method Main

   - public static void main (String[] args) {

     → Method utama program.

   - Stack<Integer> s = new Stack<Integer>();

     → Membuat objek stack bertipe Integer.

3. Operasi Push (Baris 6-8)

   s.push(42);

   s.push(-3);

   s.push(17);

→ Menambahkan elemen ke dalam stack (17 menjadi elemen teratas).

4. Menampilkan Isi Stack
   - System.out.println ("nilai stack= "+s);

     → Menampilkan seluruh isi stack.
5. Operasi Pop
   - System.out.println ("nilai pop = "+s.pop());

     → Menghapus dan menampilkan elemen teratas (17).
   - System.out.println ("nilai stack stelah pop= "+s);

     → Menampilkan isi stack setelah elemen teratas dihapus.
6. Operasi Peek
   - System.out.println ("nilai peek"+s.peek());

     → Menampilkan elemen teratas tanpa menghapusnya (-3).
   - System.out.println ("nilai stack setelah peek= "+s);

     → Menampilkan isi stack yang tetap sama setelah *peek*.

Dari Langkah-langkah diatas kita akan mendapatkan output seperti gambar 2.4 dibawah ini.

```
<terminated> latihanStack_2511533014 [Java Ap
nilai stack= [42, -3, 17]
nilai pop = 17
nilai stack stelah pop= [42, -3]
nilai peek-3
nilai stack setelah peek= [42, -3]
```

Gambar 2.4 Output kode program latihanStack

### 2.3.1 Analisis

Output tersebut menunjukkan bahwa program berhasil menjalankan operasi dasar pada struktur data Stack sesuai dengan prinsip Last In First Out (LIFO). Pada awalnya, stack berisi [42, -3, 17], di mana angka 17 berada di

posisi teratas karena merupakan elemen terakhir yang dimasukkan. Saat dilakukan operasi *pop*, elemen teratas yaitu 17 dihapus dan ditampilkan, sehingga isi stack berubah menjadi [42, -3]. Selanjutnya, operasi *peek* digunakan untuk melihat elemen teratas tanpa menghapusnya, yaitu -3, dan terbukti bahwa setelah *peek*, isi stack tetap [42, -3]. Hal ini menunjukkan bahwa operasi *pop* menghapus data, sedangkan *peek* hanya membaca data tanpa mengubah isi stack, sehingga program berjalan sesuai dengan konsep Stack.

## 2.4 Praktikum "StackArray_2511533014"

```java
package pekan3_2511533014;

public class stackArray_2511533014 {
    static final int MAX = 1000;
    int top;
    int a[] = new int [MAX];
    boolean isEmpaty_2511533014()
    {
        return (top < 0);
    }
    stackArray_2511533014 ()
    {
        top = -1;
    }
    boolean push_2511533014 (int x)
    {
        if (top >= (MAX - 1)) {
            System.out.println ("Stack Overflow");
            return false;
        }
        else {
            a[++top] = x;
            System.out.println(x + " dimasukkan dalam stack");
            return true;
        }
    }
    int pop_2511533014 ()
    {
        if (top < 0) {
            System.out.println ("Stack Underflow");
            return 0;
        }
        else {
            int x = a[top--];
            return x;
```

Gambar 2.5 Kode Program praktikum stackArray

```java
        }
    }
    int peek_2511533014()
    {
        if (top < 0) {
            System.out.println("Stack Underflow");
            return 0;
        }
        else {
            int x = a[top];
            return x;
        }
    }
    void print_2511533014() {
    for(int i = top;i>-1;i--) {
        System.out.println(" "+ a[i]);
    }
    }

}
```

Gambar 2.6 Sambungan Kode program praktikum stackArray

Kode tersebut merupakan implementasi struktur data Stack menggunakan array di Java. Class stackArray_2511533014 menyediakan operasi dasar seperti *push* untuk menambah data, *pop* untuk menghapus data teratas, *peek* untuk melihat elemen teratas, serta *print* untuk menampilkan isi stack. Stack ini menggunakan array dengan kapasitas maksimum tertentu dan variabel top sebagai penunjuk posisi elemen teratas. Program juga menangani kondisi khusus seperti *Stack Overflow* (penuh) dan *Stack Underflow* (kosong).

Berikut penjelsan langkah-langkah kerjanya:

1. static final int MAX = 1000;

   → Menentukan kapasitas maksimum stack

2. int top;

   → Menyimpan indeks elemen teratas.

3. int a[] = new int [MAX];

   → Array untuk menyimpan elemen stack.

4. boolean isEmpaty_2511533014()

   → Method untuk mengecek apakah stack kosong.

5. return (top < 0);

   → Jika top < 0 berarti stack kosong.

6. Konstruktor

   - stackArray_2511533014 ()

     → Konstruktor untuk inisialisasi.

- top = -1;

  → Menandakan stack masih kosong.

7. Method Push (Menambah Data)
   - boolean push_2511533014 (int x)

     → Method untuk menambahkan elemen.
   - if (top >= (MAX - 1))

     → Mengecek apakah stack penuh.
   - System.out.println ("Stack Overflow");

     → Menampilkan pesan jika penuh.
   - a[++top] = x;

     → Menambahkan elemen ke posisi atas.
   - System.out.println(x + " dimasukkan dalam stack");

     → Menampilkan pesan berhasil.
8. Method Pop (Menghapus Data)
   - int pop_2511533014 ()

     → Method untuk menghapus elemen teratas.
   - if (top < 0)

     → Mengecek apakah stack kosong.
   - System.out.println ("Stack Underflow");

     → Menampilkan pesan jika kosong.
   - int x = a[top--];

     → Mengambil dan menghapus elemen teratas.

9. Method Peek (Melihat Data Teratas)
   - int peek_2511533014()

     → Method untuk melihat elemen teratas.
   - int x = a[top];

     → Mengambil nilai tanpa menghapus.
10. Method Print
    - void print_2511533014()

→ Method untuk menampilkan seluruh isi stack.

- for(int i = top; i > -1; i--)

  → Perulangan dari atas ke bawah.

- System.out.println(" "+ a[i]);

  → Menampilkan setiap elemen stack.

## 2.5 Praktikum "stackArrayDriver"

```
1  package pekan3_2511533014;
2
3  public class stackArrayDriver_2511533014 {
4      public static void main (String[] args) {
5          stackArray_2511533014 s = new stackArray_2511533014();
6          s.push_2511533014 (10);
7          s.push_2511533014(20);
8          s.push_2511533014(30);
9          System.out.println(s.pop_2511533014 () + " dikeluarkan dari stack");
10         System.out.println("Elemen teratas adalah :" + s.peek_2511533014());
11         System.out.println("Elemen pada stack :");
12         s.print_2511533014();
13
14     }
15
16 }
17
```

Gambar 2.7 Kode Program praktikum stackArrayDriver

Kode tersebut merupakan class driver untuk menguji implementasi Stack berbasis array pada class stackArray_2511533014. Program ini melakukan operasi dasar Stack yaitu *push* untuk menambahkan data, *pop* untuk menghapus elemen teratas, *peek* untuk melihat elemen teratas, serta *print* untuk menampilkan seluruh isi stack. Dengan data yang dimasukkan (10, 20, 30), program menunjukkan cara kerja Stack yang mengikuti prinsip *Last In First Out (LIFO).*

Berikut Penjelasan langkah-langkah kerjanya:

1. public static void main (String[] args) {

   → Method utama program.

2. stackArray_2511533014 s = new stackArray_2511533014();

   → Membuat objek stack dari class yang telah dibuat.

3. Operasi Push (Baris 6-8)

   s.push_2511533014(10);

   s.push_2511533014(20);

s.push_2511533014(30);

→ Menambahkan elemen ke dalam stack (30 menjadi elemen teratas).

4. Operasi Pop

   System.out.println(s.pop_2511533014() + " dikeluarkan dari stack");

   → Menghapus dan menampilkan elemen teratas (30).

5. Operasi Peek

   System.out.println("Elemen teratas adalah :" + s.peek_2511533014());

   → Menampilkan elemen teratas setelah pop (20).

6. Menampilkan Isi Stack
   - System.out.println("Elemen pada stack :");

     → Menampilkan judul output.
   - s.print_2511533014();

     → Menampilkan seluruh isi stack dari atas ke bawah (20 lalu 10).

Dari Langkah-langkah diatas kita akan mendapatkan output seperti gambar 2.8 dibawah ini.

```
<terminated> stackArrayDriver_2511533014 [
10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30 dikeluarkan dari stack
Elemen teratas adalah :20
Elemen pada stack :
 20
 10
```

Gambar 2.8 Output kode program stackArrayDriver

### 2.5.1 Analisis

Output tersebut menunjukkan bahwa program Stack berbasis array telah berjalan dengan benar sesuai konsep Last In First Out (LIFO). Pada awalnya, data 10, 20, dan 30 berhasil dimasukkan ke dalam stack, yang ditandai dengan pesan “dimasukkan dalam stack”, di mana 30 menjadi elemen teratas. Ketika dilakukan operasi *pop*, elemen teratas yaitu 30 dikeluarkan dari stack. Setelah itu, operasi *peek* menampilkan elemen teratas

yang baru yaitu 20 tanpa menghapusnya. Terakhir, saat seluruh isi stack ditampilkan, terlihat bahwa elemen yang tersisa adalah 20 dan 10, dengan urutan dari atas ke bawah. Hal ini membuktikan bahwa semua operasi (*push, pop, peek, dan print*) berjalan dengan baik dan sesuai dengan logika Stack.

## 2.6 Praktikum “StackPostfix_2511533014”

```java
package pekan3_2511533014;
import java.util.Scanner;

public class StackPostfix_2511533014 {
    public static int postfixEvaluate_2511533014 (String expression) {
        Stack<Integer> s = new Stack<Integer>();
        Scanner input = new Scanner (expression);
        while (input.hasNext()) {
            if (input.hasNextInt()) { // an operand (integer)
                s.push(input.nextInt());
            } else {
                String operator = input.next();
                int operand2 = s.pop();
                int operand1 = s.pop();
                if (operator.equals("+")) {
                    s.push(operand1 + operand2);
                } else if (operator.equals("-")) {
                    s.push(operand1 - operand2);
                } else if (operator.equals("*")) {
                    s.push(operand1 * operand2);
                } else {
                    s.push(operand1 / operand2);
                }
            }

        }
//      input.close();
        return s.pop();
    }
    public static void main(String[] args) {
        System.out.println("hasil postfix= " + postfixEvaluate_2511533014("5 2 4 * + 7 -"));
    }

            }
```

Gambar 2.9 Kode Program Praktikum StackPostfix

Kode tersebut merupakan program Java yang digunakan untuk menghitung hasil *ekspresi postfix (Reverse Polish Notation)* menggunakan struktur data *Stack.* Method postfixEvaluate_2511533014 membaca ekspresi dalam bentuk string, kemudian memproses setiap token (angka atau operator). Jika token berupa angka, maka dimasukkan ke dalam stack, sedangkan jika berupa operator, program akan mengambil dua angka teratas dari stack, melakukan operasi sesuai operator, lalu menyimpan kembali hasilnya ke dalam stack. Pada method main, program menghitung ekspresi "5 2 4 * + 7 -" dan menampilkan hasil akhirnya.

Berikut penjelasan langkah-langkah kerjanya:

1. importjava.util.Scanner;

   import java.util.Stack;

   → Mengimpor Scanner dan Stack.

2. Method Evaluasi Postfix

   - public static int postfixEvaluate_2511533014 (String expression) {

     → Method untuk menghitung ekspresi postfix.

   - Stack<Integer> s = new Stack<Integer>();

     → Membuat stack untuk menyimpan angka.

   - Scanner input = new Scanner (expression);

→ Membaca ekspresi sebagai input token per token.

3. Proses Token
   - while (input.hasNext()) {

     → Perulangan selama masih ada token.
   - if (input.hasNextInt()) {

     → Mengecek apakah token adalah angka.
   - s.push(input.nextInt());

     → Jika angka, masukkan ke stack.
4. String operator = input.next();

   int operand2 = s.pop();

   int operand1 = s.pop();

   → Mengambil operator dan dua operand dari stack.
5. Proses Operasi
   - + → penjumlahan
   - - → pengurangan
   - * → perkalian

     Hasil operasi dimasukkan kembali ke stack.
6. Penutup Method
   - return s.pop();

     → Mengembalikan hasil akhir.
7. Method Main (Baris 26-28)
   - System.out.println("hasil postfix= " + postfixEvaluate_2511533014("5 2 4 * + 7 -"));

     → Menjalankan evaluasi postfix dan menampilkan hasil.
8. Alur Perhitungan Ekspresi

   Ekspresi: 5 2 4 * 7 –
   - 2 * 4 = 8
   - 5 + 8 = 13
   - 13 – 7 = 6

Dari Langkah-langkah diatas kita akan mendapatkan output seperti gambar 2.10 dibawah ini.

```
<terminated> StackPostfix_2511533014 [
hasil postfix= 6
```

Gambar 2.10 Output Kode program StackPostfix

### 2.6.1 Analisis

Output “hasil postfix = 6” menunjukkan bahwa program berhasil mengevaluasi ekspresi postfix "5 2 4 * + 7 -" dengan benar menggunakan struktur data Stack. Proses perhitungan dilakukan secara bertahap, dimulai dari operasi perkalian 2 * 4 = 8, kemudian hasilnya dijumlahkan dengan 5 menjadi 13, dan terakhir dikurangi 7 sehingga diperoleh hasil akhir 6. Hal ini membuktikan bahwa algoritma evaluasi postfix yang diimplementasikan sudah berjalan sesuai dengan konsep Stack, di mana operand disimpan terlebih dahulu dan operator diproses dengan mengambil dua elemen teratas dari stack.

## 2.7 Praktikum “SiswaStack_2511533014”

```java
package pekan3_2511533014;
import java.util.ArrayList;
class Siswa_2511533014 {
    String nama;
    int nim;

    public Siswa_2511533014(String nama, int nim) {
        this.nama = nama;
        this.nim = nim;
    }

    @Override
    public String toString() {
        return "Nim: " + nim + ", Nama: " + nama;
    }
}

public class SiswaStack_2511533014 {
    private ArrayList<Siswa_2511533014> stack;

    public SiswaStack_2511533014() {
        stack = new ArrayList<>();
    }

    public void push_2511533014(Siswa_2511533014 mhs) {
        stack.add(mhs);
    }

    public Siswa_2511533014 pop_2511533014() {
        if (!isEmpty_251153014()) {
            return stack.remove(stack.size() - 1);
        }
        return null;
    }
```

Gambar 2.11 Kode Program Praktikum SiswaStack

```
36    public Siswa_2511533014 peek_2511533014() {
37        if (!isEmpty_251153014()) {
38            return stack.get(stack.size() - 1);
39        }
40        return null;
41    }
42
43    public boolean isEmpty_251153014() {
44        return stack.isEmpty();
45    }
46
47    public void tampilkanSiswa_2511533014() {
48        for (int i = stack.size() - 1; i >= 0; i--) {
49            System.out.println(stack.get(i));
50        }
51    }
52
53    public static void main(String[] args) {
54        SiswaStack_2511533014 studentStack = new SiswaStack_2511533014();
55
56        Siswa_2511533014 mhs1 = new Siswa_2511533014("Ali", 1);
57        Siswa_2511533014 mhs2 = new Siswa_2511533014("Boby", 2);
58        Siswa_2511533014 mhs3 = new Siswa_2511533014("Charles", 3);
59
60        studentStack.push_2511533014(mhs1);
61        studentStack.push_2511533014(mhs2);
62        studentStack.push_2511533014(mhs3);
63
64        System.out.println("Siswa di dalam stack:");
65        studentStack.tampilkanSiswa_2511533014();
66
67        System.out.println("Siswa teratas: " + studentStack.peek_2511533014());
68        System.out.println("Mengeluarkan siswa: " + studentStack.pop_2511533014());
69
70        System.out.println("Setelah pop:");
```

Gambar 2.12 Sambungan Kode Program Praktikum SiswaStack

```
70        System.out.println("Setelah pop:");
71        studentStack.tampilkanSiswa_2511533014();
72    }
73 }
```

Gambar 2.13 Sambungan Kode Program Praktikum SiswaStack

Kode program tersebut merupakan implementasi struktur data Stack menggunakan ArrayList untuk menyimpan objek bertipe Siswa. Class Siswa_2511533014 digunakan untuk merepresentasikan data mahasiswa (nama dan NIM), sedangkan class SiswaStack_2511533014 berfungsi sebagai stack dengan operasi *push*, *pop*, *peek*, dan *isEmpty*. Program pada method main menambahkan beberapa data siswa ke dalam stack, menampilkan isi stack, melihat elemen teratas, menghapus elemen teratas, lalu menampilkan kembali isi stack setelah perubahan, sesuai dengan prinsip *Last In First Out (LIFO).*

Berikut Penjelasan langkah-langkah kerjanya:

1. Stringnama;

   int nim;

   → Atribut untuk menyimpan nama dan NIM.

2. Baris 7-10

   Konstruktor untuk menginisialisasi objek siswa.

3. Baris 12-15

toString()

→ Mengatur format output saat objek ditampilkan.

4. Class Stack
    - public class SiswaStack_2511533014 {

      → Class utama stack.
    - private ArrayList<Siswa_2511533014> stack;

      → Struktur data untuk menyimpan elemen stack.
5. Konstruktor (Baris 20-22)

   → Menginisialisasi ArrayList.
6. Method Push (Baris 24-26)

   push_2511533014()

   → Menambahkan data ke stack (ke posisi terakhir).
7. Method Pop (Baris 28-33)

   → Menghapus dan mengembalikan elemen teratas jika stack tidak kosong.
8. Method Peek (Baris 35-40)

   → Mengambil elemen teratas tanpa menghapus.
9. Method isEmpty (Baris 42-44)

   → Mengecek apakah stack kosong.
10. Method Tampilkan (Baris 46-50)

    → Menampilkan isi stack dari atas ke bawah.

11. Method Main (Driver)

    - Baris 52-54

      → Membuat objek stack.
    - Baris 56-58

      → Membuat 3 objek siswa: Ali, Boby, Charles.
    - Baris 60-62

      → Menambahkan siswa ke stack.
    - Baris 64-65

      → Menampilkan seluruh isi stack.

- Baris 67

  → Menampilkan elemen teratas (*peek*).
- Baris 68

  → Menghapus elemen teratas (*pop*).
- Baris 70-71

  → Menampilkan isi stack setelah penghapusan.

Dari Langkah-langkah diatas kita akan mendapatkan output seperti gambar 2.14 dibawah ini.

```
<terminated> SiswaStack_2511533014 [Java Application] C
Siswa di dalam stack:
Nim: 3, Nama: Charles
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
Siswa teratas: Nim: 3, Nama: Charles
Mengeluarkan siswa: Nim: 3, Nama: Charles
Setelah pop:
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
```

Gambar 2.14 Output Kode program SiswaStack

### 2.7.1 Analisis

Output tersebut menunjukkan bahwa program berhasil mengimplementasikan struktur data Stack berbasis objek menggunakan ArrayList dengan prinsip Last In First Out (LIFO). Pada awalnya, tiga data siswa dimasukkan ke dalam stack yaitu Ali, Boby, dan Charles, sehingga saat ditampilkan, urutan dimulai dari yang paling atas yaitu Charles, kemudian Boby, dan Ali. Hal ini menunjukkan bahwa elemen terakhir yang dimasukkan berada di posisi teratas. Selanjutnya, operasi *peek* menampilkan siswa teratas yaitu Charles tanpa menghapusnya. Kemudian, operasi *pop* menghapus siswa teratas tersebut, yaitu Charles, dari stack. Setelah dilakukan *pop*, isi stack ditampilkan kembali dan hanya menyisakan Boby dan Ali. Dengan demikian, seluruh operasi stack (*push, peek, dan pop*) berjalan dengan baik sesuai dengan konsep LIFO.

## 2.8 Latihan Praktikum

### 2.8.1 Latihan "Website_2511533014"

```java
package pekan3_2511533014;
public class Website_2511533014 {

        private String judul_3014;
        private String url_3014;
        // Constructor
        public Website_2511533014(String judul, String url) {
            this.judul_3014 = judul;
            this.url_3014 = url;
        }
        // Getter
        public String getJudul_3014() {
            return judul_3014;
        }

        public String getUrl_3014() {
            return url_3014;
        }

        // Setter
        public void setJudul_3014(String judul) {
            this.judul_3014 = judul;
        }

        public void setUrl_3014(String url) {
            this.url_3014 = url;
        }

        // toString untuk menampilkan data
        @Override
        public String toString() {
            return "Judul: " + judul_3014 + "\nURL: " + url_3014;
        }
    }

```

Kode tersebut merupakan class Website_2511533014 yang digunakan untuk merepresentasikan data sebuah website dengan atribut judul dan URL. Class ini memiliki konstruktor untuk menginisialisasi data, method *getter* dan *setter* untuk mengakses serta mengubah nilai atribut, serta method toString() yang berfungsi untuk menampilkan informasi website dalam format yang rapi. Class ini biasanya digunakan sebagai objek dalam struktur data seperti Stack atau ArrayList.

Berikut Penjelasan langkah-langkah kerjanya:

1. private String judul_3014;

   private String url_3014;

   → Atribut untuk menyimpan judul dan alamat website.

2. Konstruktor

    - public Website_2511533014(String judul, String url)

      → Konstruktor untuk menginisialisasi objek saat dibuat.
    - this.judul_3014 = judul;

      → Mengisi nilai judul.

    - this.url_3014=url;

      → Mengisi nilai URL.
3. Getter (Baris 12-16)

   getJudul_3014() dan getUrl_3014()

   → Mengembalikan nilai judul dan URL.
4. Setter (Baris 18-24)

   setJudul_3014() dan setUrl_3014()

   → Mengubah nilai judul dan URL.
5. Method toString

    - toString()

      → Mengatur tampilan output objek saat dicetak.
    - return "Judul: " + judul_3014 + "\nURL: " + url_3014;

      → Menampilkan judul dan URL dalam format rapi.

### 2.8.2 Latihan “Browser_2511533014”

```java
package pekan3_2511533014;
import java.util.Scanner;
import java.util.Stack;

public class Browser_2511533014 {
        public static void main(String[] args) {
            Scanner sc = new Scanner(System.in);
            Stack<Website_2511533014> history = new Stack<>();
            int pilihan;

            do {
                System.out.println("\n=== Browser History NIM: 2511533014 ===");
                System.out.println("1. Kunjungi Website (Push)");
                System.out.println("2. Tombol Back (Pop)");
                System.out.println("3. Lihat Halaman Aktif (Peek)");
                System.out.println("4. Keluar");
                System.out.print("Pilihan: ");
                pilihan = sc.nextInt();
                sc.nextLine();

                switch (pilihan) {

                    // PUSH
                    case 1:
                        System.out.print("Masukkan Judul: ");
                        String judul = sc.nextLine();

                        System.out.print("Masukkan URL: ");
                        String url = sc.nextLine();

                        history.push(new Website_2511533014(judul, url));
                        System.out.println("Berhasil mengunjungi halaman!");
                        break;
```

```java
                    // POP
                    case 2:
                        if (!history.isEmpty()) {
                            Website_2511533014 keluar = history.pop();
                            System.out.println("Kembali dari halaman:");
                            System.out.println(keluar.getJudul_3014());
                        } else {
                            System.out.println("History kosong, tidak bisa kembali.");
                        }
                        break;

                    // PEEK
                    case 3:
                        if (!history.isEmpty()) {
                            System.out.println("Halaman saat ini:");
                            System.out.println(history.peek());
                        } else {
                            System.out.println("Belum ada halaman yang dibuka.");
                        }
                        break;

                    // KELUAR
                    case 4:
                        System.out.println("Keluar dari program.");
                        break;

                    default:
                        System.out.println("Pilihan tidak valid.");
                }

            } while (pilihan != 4);

            sc.close();
        }
    }
```

Program ini merupakan implementasi struktur data Stack (LIFO) dalam Java untuk mensimulasikan browser history. Setiap halaman website yang dikunjungi akan dimasukkan ke dalam stack menggunakan operasi *push*, sehingga halaman terakhir berada di posisi paling atas. Ketika pengguna menekan tombol *back*, program akan menghapus halaman terakhir menggunakan *pop*. Selain itu, program juga menyediakan fitur untuk melihat

halaman aktif menggunakan *peek* tanpa menghapusnya. Seluruh proses dikendalikan melalui menu interaktif dan program juga menangani kondisi stack kosong agar tidak terjadi error.

Berikut penjelasan langkah-langkah kerjanya:

1. Scanner sc = new Scanner(System.in);
   → Membuat objek Scanner untuk input.
2. Stack<Website_2511533014> history = new Stack<>();
   → Membuat stack untuk menyimpan data website.
3. Int pilihan;
   → Variabel untuk menyimpan pilihan menu.
4. do {
   → Memulai perulangan menu.
5. System.out.println("\n=== Browser History NIM: 2511533014 ===");
   System.out.println("1. Kunjungi Website (Push)");
   System.out.println("2. Tombol Back (Pop)");
   System.out.println("3. Lihat Halaman Aktif (Peek)");
   System.out.println("4. Keluar");
   → Menampilkan menu pilihan.
6. System.out.print("Pilihan: ");
   → Meminta input pengguna.
7. pilihan = sc.nextInt();
   → Membaca pilihan menu.
8. sc.nextLine();
   → Membersihkan newline agar input berikutnya tidak error.
9. switch (pilihan) {

   → Menentukan aksi berdasarkan pilihan user.

10. System.out.print("Masukkan Judul: ");
    String judul = sc.nextLine();
    → Input judul website.
11. System.out.print("Masukkan URL: ");
    String url = sc.nextLine();
    → Input URL website.

12. history.push(new Website_2511533014(judul, url));
    → Menambahkan data ke stack.

13. System.out.println("Berhasil mengunjungi halaman!");
    → Menampilkan pesan sukses.
14. if (!history.isEmpty()) {
    → Mengecek apakah stack tidak kosong.
15. Website_2511533014 keluar = history.pop();
    → Menghapus data terakhir.
16. System.out.println("Kembali dari halaman:");

    System.out.println(keluar.getJudul_3014());

    → Menampilkan halaman yang dihapus.

17. **Baris 40–42**
    → Jika kosong, tampilkan pesan error.
18. if (!history.isEmpty()) {
    → Cek apakah ada data.
19. System.out.println("Halaman saat ini:");

  System.out.println(history.peek());

→ Menampilkan halaman teratas tanpa menghapusnya.

20. **Baris 50–52**
    → Jika kosong, tampilkan pesan.
21. System.out.println("Keluar dari program.");
    → Menampilkan pesan keluar.
22. System.out.println("Pilihan tidak valid.");
    → Menampilkan pesan jika input salah.
23. } while (pilihan != 4);
    → Program akan terus berjalan sampai pilih keluar.
24. sc.close();
    → Menutup Scanner.

Dari Langkah-langkah diatas kita akan mendapatkan output seperti gambar dibawah ini.

```
Browser_2511533014 [Java Application] C:\Users\rara0\.p2\po

=== Browser History NIM: 2511533014 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 1
Masukkan Judul: SSO UNAND
Masukkan URL: https://sso.unand.ac.id/login
Berhasil mengunjungi halaman!
```

### 2.8.3 Analisis

Output tersebut menunjukkan proses penggunaan struktur data Stack (LIFO) pada simulasi browser history. Pada tahap awal, program menampilkan menu utama dan pengguna memilih opsi 1 (Kunjungi Website / Push). Selanjutnya pengguna memasukkan data website berupa judul *SSO UNAND* dan URL *https://sso.unand.ac.id/login*. Data tersebut kemudian dimasukkan ke dalam stack menggunakan operasi *push*, sehingga halaman tersebut menjadi elemen teratas (top) dalam stack dan sekaligus menjadi halaman aktif yang sedang dibuka. Keberhasilan proses ini ditandai dengan munculnya pesan *"Berhasil mengunjungi halaman!"*.

Secara keseluruhan, output ini menunjukkan bahwa proses penambahan data ke dalam stack telah berjalan dengan benar sesuai konsep LIFO, di mana setiap halaman yang baru dikunjungi akan berada di posisi paling atas dan siap untuk diakses atau dihapus terlebih dahulu pada operasi berikutnya.

# BAB III
# PENUTUP

## 3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Stack merupakan struktur data yang sederhana namun sangat penting dalam pemrograman. Dengan prinsip kerja *Last In First Out (LIFO)*, Stack memungkinkan pengelolaan data secara terstruktur dan efisien. Operasi dasar seperti *push*, *pop*, dan *peek* dapat diimplementasikan dengan baik dalam program Java. Secara keseluruhan, penggunaan Stack sangat membantu dalam menyelesaikan berbagai permasalahan yang berkaitan dengan urutan data dan proses yang bersifat berlapis.

## DAFTAR PUSTAKA

[1] Oracle, "Stack Class (Java Platform SE)," [Online]. Available: https://docs.oracle.com/javase/8/docs/api/java/util/Stack.html

[2] GeeksforGeeks, "Stack Data Structure," [Online]. Available: https://www.geeksforgeeks.org/stack-data-structure/

[3] W3Schools, "Java Stack," [Online]. Available: https://www.w3schools.com/java/java_stack.asp

[4] Tutorialspoint, "Data Structure - Stack," [Online]. Available: https://www.tutorialspoint.com/data_structures_algorithms/stack_algorithm.htm